Towards Perfect Code Generation: Iterative API-based Feedback Loops

Harman Preet Singh
852863316

Abstract

The abstract paragraph should be indented ½ inch (3 picas) on both the left- and right-hand margins. Use 10 point type, with a vertical spacing (leading) of 11 points. The word **Abstract** must be centered, bold, and in point size 12. Two line spaces precede the abstract. The abstract must be limited to one paragraph.

Keywords: keyword 1 · keyword 2 · keyword 3

1 Introduction

Large Language Models (LLMs) are increasingly used in the field of software development for the generation of code, including HTML. These models are very capable of producing syntactically valid HTML that renders correctly in web browsers. However, there is a significant blind spot: while the generated code may be syntactically correct, it often lacks semantic correctness. In other words, the code functions as intended, but it may not adhere to best practices.

This potential non-adherence to semantic best practices is often seen in the overuse of the `<div>` tag, a phenomenon colloquially referred to as "divitis". Instead of using semantically meaningful tags in layout and content structure, such as `<section>` to denote sections or `<article>` for articles, developers may default to using `<div>` tags. For example, a developer might wrap an entire article in a `<div>` instead of using the more appropriate `<article>` tag. LLMs are trained on real code snippets, which often contain this pattern, making them likely to reproduce it in their generated output. Additionally, if a codebase already contains a significant amount of "divitis" code-generation tools like Claude Code may produce new code that perpetuates this issue.

There are two real consequences. Code that is hard to maintain, as the semantic meaning of the content has become an afterthought. But more importantly, users with visual impairments who rely on screen readers are negatively affected because screen readers use the semantic structure of a webpage to navigate them. A lack of semantic structure can make it difficult for these users to understand the content and context of a page.

Instructing an LLM to use semantic tags through a system prompt or explicit instructions within the prompt itself does not guarantee compliance, particularly for smaller or local models. There is a need for a reliable automated mechanism that can catch and correct this class of semantic errors in LLM-generated HTML.

We propose a solution in the form of an iterative validation-and-regeneration feedback loop. In this approach, the LLM-generated HTML is first validated using a validator tool which can flag semantic issues. The LLM is then reprompted with the identified errors, allowing it to correct its output. This loop continues until the output is free of errors or a maximum number of iterations is reached.

In this report, we aim to find out (RQ1) to what extent an iterative feedback loop can improve the semantic correctness of LLM-generated HTML code, and (RQ2) whether this guardrail approach can enable smaller, cheaper LLMs to achieve semantic correctness comparable to larger models. The remainder of this report is structured as follows: Section 2 reviews related work, Section 3 presents the research questions and hypotheses, Section 4 details the methodology, Section 5 describes the experimental setup, Section 6 presents the results, and Section 7 discusses the findings and implications. Finally, we conclude with a summary of our contributions and potential future work.

2 Related Work

Related work spans two interconnected areas: (1) LLM code generation quality, and (2) iterative self-correction and feedback loops. In the first part, we discuss what the literature says about LLMs generating code: their strengths, but also the documented tendency to reproduce patterns from training data including bad practices. In the second part, we cover approaches where LLMs are given feedback and asked to revise their output, particularly for Answer Set Programming (ASP) code. Finally, we review what is known about HTML semantic quality as a measurable property and the use of automated validation as a guardrail.

The literature on LLM code generation highlights both the strengths and limitations of these models. While LLMs can autonomously generate functioning code, they are also prone to reproducing patterns from their training data, including suboptimal or even harmful coding practices. For instance, Souly et al. [1] demonstrate that LLMs can be susceptible to poisoning attacks, where maliciously crafted inputs lead to the generation of flawed code. They show that even a small amount of poisoned data can significantly degrade the model's performance.

Tambon et al. [2] examined samples of 333 bugs collected from code generated using three leading LLMs and identified 10 common bug patterns, not including our focus on semantic quality. Dou et al. [3] evaluated several models on the length and complexity of generated code, finding that LLMs face challenges in generating successful code for more complex problems. They propose a novel training-free iterative method where the model gains the ability to self-critique and correct its own code, resulting in a repair success rate of 29.2% after two iterations. This study particularly is very relevant to our work, as it demonstrates the potential of iterative feedback loops in improving code quality. However, it primarily focuses on functional correctness rather than semantic or stylistic correctness, which is a gap that our research aims to address.

Addressing the second area of related work, several studies have explored iterative feedback and self-correction loops for LLMs. Ravi et al. [4] present a method that employs multiple feedback models to iteratively refine code generated by LLMs. Their approach demonstrates that iterative feedback can significantly enhance code quality, but it also introduces potential biases from the feedback models themselves. In contrast, our approach does not use other LLMs as feedback sources; instead, we employ an external, deterministic validator to provide feedback. This choice mitigates the risk of introducing additional biases and allows for a more objective assessment of code quality.

Madaan et al. [5] introduce 'Self-Refine', a self-refinement framework where the model critiques its own output and iteratively improves it. They tested the framework out on 7 diverse tasks, ranging from dialog response generation to mathematical reasoning, and showed that the generated code improves by 20% on average in task performance. The proposed method uses a single LLM as the generator, refiner and the feedback provider, whereas our approach separates the feedback mechanism from the generation process.

Much research has been conducted comparing the performance of smaller versus larger LLMs on code generation tasks. Coignion et al. [6] show that larger models generally outperform smaller ones in terms of code quality and functional correctness. However, they note that actual runtime execution efficiency of the generated code does not vastly differ, that is, smaller models can still produce code that runs efficiently.

Historically, treating semantic correctness as a measureable, objective property has been difficult because 'meaning' is highly contextual. However, some prior work has attempted to quantify semantic correctness in code. One example is the 'Generic-to-Semantic Ratio' (R_{gs}), which measures the count of non-semantic structural nodes (`<div>`, `<span>`) against native layout elements (`<header>`, `<nav>`, `<main>`, `<section>`, `<footer>`). A lower ratio indicates higher structural semantic quality. The 'Heading Hierarchy Traversal' (H_{score}) method measures structural correctness by scanning a webpage for skipped heading levels (e.g., a jump from `<h1>` directly to `<h3>`, skipping `<h2>`). These approaches provide a way to quantify semantic quality, but they are limited in scope and do not capture the full semantic intent of the code. In comparison, the W3C Markup Validation Service is a widely used tool that checks for syntactic and structural conformance to HTML standards. It is capable of identifying errors such as unclosed tags, incorrect nesting, and invalid attributes. However, it does not assess the semantic intent of the code, which is a critical aspect of web accessibility and user experience. This gap highlights the need for more comprehensive evaluation methods that can capture both syntactic correctness and semantic quality in HTML code. Our work does not address this gap, instead we acknowledge the need for further research in this area.

No prior work combines an external deterministic validator, an iterative reprompting loop, and a cost-efficiency comparison across model sizes in the context of HTML semantic quality. Our research aims to fill this gap by providing a comprehensive evaluation of how iterative feedback loops can improve the semantic quality of code generated by LLMs. By using an external validator, we can objectively assess the quality of the generated code without introducing additional biases. Additionally, our study will explore the cost-efficiency of using smaller models in comparison to larger ones.

3 Research Questions & Hypotheses

We aim to answer two research questions:

1. **RQ1:** Does the iterative API-based feedback loop improve the quality of code generation? We operationalise "quality" as the reduction in the number of errors in the generated code, measured before and after applying the feedback loop.
2. **RQ2:** Does this guardrail approach enable smaller, cheaper LLMs to achieve comparable quality to larger, more expensive models? We define "comparable quality" as equivalent error reduction rates, and "cost" is measured via mean generation time per prompt per model. Cloud-based models are explicitly out of scope for this study, as all models are run locally through Ollama, ensuring consistent evaluation conditions and avoiding external dependencies.

To evaluate our feedback loop, we also test the models under a blind condition, where the regeneration is performed without detailed feedback. This allows us to isolate the effect of the feedback loop.

We expect the feedback condition to outperform the blind condition across all models, as the iterative feedback loop provides targeted guidance for error correction. Furthermore, we hypothesise that reasoning-capable models (i.e., models that think and reason through the problem before generating code) will converge faster regardless of parameter count. Additionally, we expect that difficult prompts will require more iterations to achieve error-free code generation.

4 Methodology

4.1 System Architecture

Introduce the three pillars of the system: the LLM (served locally via Ollama), the 'Nu Html Checker (vnu)' service (hosted in Docker [1]), and the Python orchestration pipeline. Explain that these are deliberately kept modular – the LLM and validator are interchangable without rewriting the pipeline. This modularity is a design choice worth stating explicitly. Mention that we will refer to the validator as 'the validator'.

Walk through one full iteration of the loop: prompt -> LLM generates HTML -> validator checks it -> if errors exist and the iteration limit is not reached -> reprompt with original code + error list -> repeat. Reference your flowchart figure here. Keep it process-focused, not implementation-focused.

Justify why you run the validator locally in Docker rather than hitting the public API. Two reasons: avoiding rate limits during large-scale batch runs across 51 prompts x multiple models x multiple conditions, and ensuring the validator version is pinned and consistent across all runs. The latter matters for reproducibility.

4.2 Prompt Dataset

Explain that the prompt set was hand-crafted rather than sampled from an existing benchmark, because no established benchmark exists for HTML semantic quality specifically. The goal was to create tasks that structurally require semantic tags – so that a model defaulting to `<div>` produces objectively worse output. This justifies the custom dataset.

Describe the tiered structure: simple (single-component pages), medium (multi-section layouts), difficult (complex interactive or multi-role pages). Explain what makes a prompt "harder" – more opportunities for semantic tags, more structural complexity, higher chance of divitis. Give one or two illustrative examples per tier without listing all 51.

Explain that all models are evaluated on the exact same 51 prompts in the same order. This fixed design ensures that differences in results are attributable to the model or condition, not to prompt sampling variance. Constrast this briefly with a random-sampling approach and why you rejected it. (how I initially did it)

4.3 Models & Configurations

Explain why you chose the models you did. The selection spans a range of parameter counts and a simple binary range of reasoning capabilities, which is deliberate – you want both axes to vary so you can disentangle their effects. Mention that all models are available through Ollama and run locally, which keeps the hardware environment constant.

Introduce reasoning capability as an explicit experimental variable, distinct from parameter count. Flag the key constrast: codestral:22b has the most parameters but lacks native reasoning, while smaller reasoning models

[1] ghcr.io/validator/validator:latest

outperform it. This is not a post-hoc observation – it was an emerging finding from initial testing that shaped the final model selection. Present a model summary table here (model name, parameter count, reasoning: yes/no, quantisation -> always no for us).

Explain that the maximum iteration count is itself a parameter of interest. You run each (model, prompt, condition) combination up to the maximum, recording when convergence occurs. This lets you analyse diminishing returns and determine the practical cut-off beyond which additional iterations yield negligible improvements – which feeds directly into the cost-quality tradeoff in RQ2.

4.4 Validation Strategy

Be transparent upfront: the W3C Markup Validation Service checks structural and syntactic conformance to the HTML specification. It does not evaluate semantic intent – it cannot tell you that `<div class="nav">` is worse than `<nav>`. State this limitation clearly, because it is the central methodological tension of your study.

Despite this, the validator is a meaningful signal. Many divitis-related errors manifest as structural issues (missing landmark elements, improper nesting, attribute misuse) that the validator does flag. Argue that error/warning count reduction is a necessary but not sufficient proxy for semantic improvement – useful, but it must be triangulated with other measures.

Introduce the blind condition as a deliberate experimental control. For every (model, prompt) pair, the pipeline runs two conditions in parallel: the `feedback` condition (the model receives the validator's specific error list) and the `blind` condition (the model is simply told to review and improve its output, with no error details). The difference in outcomes between these conditions isolates the contribution of the validator feedback itself, distinguishing it from the general benefit of giving a model a second attempt.

Explain the role of `check_validator_determinism.py`. Before trusting any before/after deltas, you verify that the validator produces identical results when run on the same HTML input multiple times. This is a non-trivial validity check – if the validator were stochastic, your improvement metrics would be meaningless. State the result of this check (presumably: the validator is deterministic).

4.5 Cost Metric

Explain that since all moedls run locally via Ollama, there is no API pricing to measure. Instead, cost is proxied through three values captured per iteration: `gen_time_s` (wall-clock generation time), `prompt_eval_count` (tokens in the prompt), and `eval_count` (tokens generated). These together give a picture of both time cost and computational load per run.

Acknowledge explicitly that no cloud-hosted large model (e.g. GPT-4o, Claude) is included in this study. The comparison is therefore entirely within the local model ecosystem. State this as a deliberate scope boundary – cloud API costs introduce pricing variables outside your control – and note it as a direction for future work. This is an honest limitation rather than a flaw, as long as it is named.

5 Experimental Setup

Prior to the full experimental run, a small-scale pilot was conducted on a subset of prompts using a small variety of models. The most striking observation from this pilot was that models with native reasoning capabilities consistently resolved all validator errors within a single iteration, whereas non-reasoning models did not. Such models often were not able to remove all errors within the pre-defined iteration limit of 3. This finding directly informed the decision to treat reasoning capability as an explicit variable in the final design, rather than focusing solely on parameter count. Furthermore, the pilot revealed a practical issue with local model outputs: models wrapped their HTML outputs in markdown code fences due to how these models are trained to generate outputs. In graphical user interfaces, the generated markdown content is automatically rendered correctly, but for our experiment – where we want to directly judge the output – this would cause issues. Occasionally the models would prefix responses with explanatory prose, which the validator would reject as incorrect HTML syntax. For this reason, for the full experimental run we introduce a module which strips such formatting artefacts before validation so that presentation-level slip-ups are not conflated with HTML errors. To conclude, the pilot served a dual purpose: it validated the core loop design and revealed the preprocessing steps necessary for a fair before/after comparison

6 Results

6.1 RQ1 — Guardrail Effectiveness

Before presenting any numbers, restate the key distinction from Section 4.4: W3C error/warning counts are your primary metric, but they are a proxy for semantic correctness, not a direct measure of it. Flag this explicitly here so the reader interprets what follows with the right expectations. One short paragraph, load-bearing for the integrity of the section.

Present the aggregate picture first: total error and warning counts before and after the guardraila cross all models and all 51 prompts, in the `feedback` condition. Introduce your primary summary table here (errors before/after, warnings before/after, delta, percentage reduction). State the result of the paired Wilcoxon test and 95% confidence interval – does the guardrail produce a statistically significant reduction in errors?

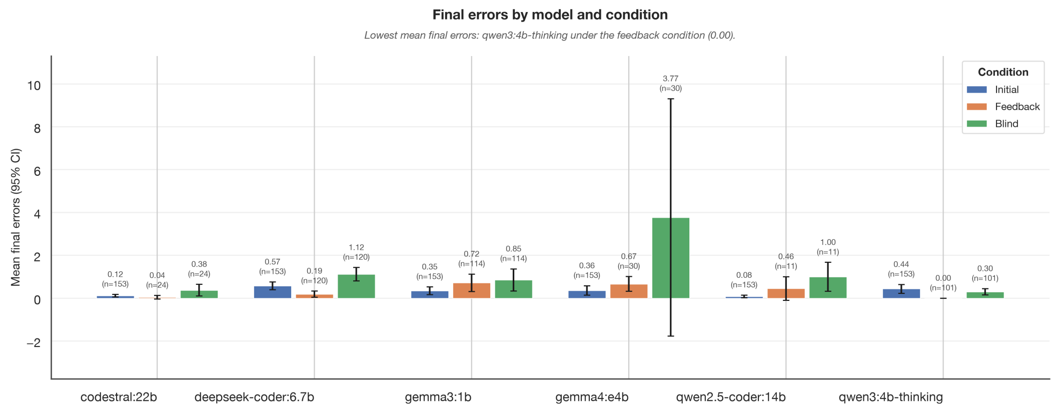

Figure 1: Mean final error count (95% CI) by model and condition (initial / feedback / blind). Value and n are annotated above each bar since several models converge to exactly zero.

Now introduce the ablation comparison. How does the `feedback` condition compare to the `blind` condition in terms of error reduction? This is the central validity argument for your guardrail design. If the `feedback` condition significantly outperforms `blind`, it means the validator's specific error list is doing real work, not just the act of reprompting. Present this as a table or figure comparing mean error reduction across both conditions, with statistical test results. I think I can split this up by difficulty of the prompt, for reasoning and non-reasoning models combined (2 categories) -> so 2 columns and 6 rows.

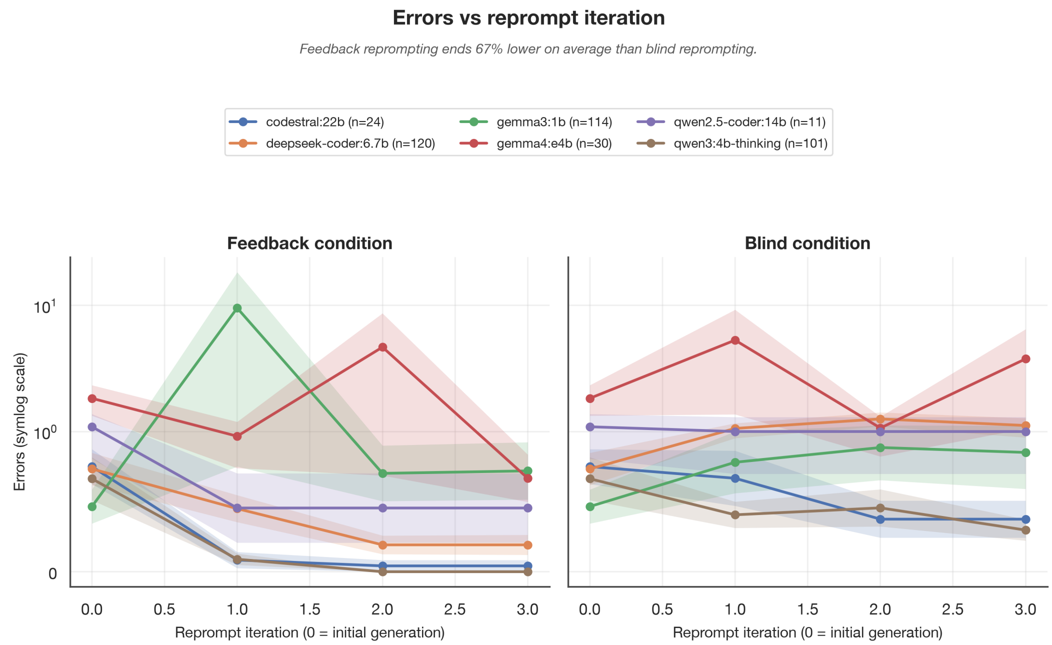

Figure 2: Mean error count per reprompt iteration, one line per model, feedback vs blind condition (shaded band = ± 1 SEM; symlog y-axis since per-model means span roughly 0–60 errors). Feedback and blind start from the same iteration-0 (initial) generation.

Discuss whether error reduction varies across simple / medium / difficult prompts. You would expect harder prompts to have higher starting error counts (because they simply have more lines of code to make a mistake in) and potentially require more iterations to converge – confirm or challenge this expectation with your data. A grouped bar chart per difficulty tier (before/after, per condition) works well here.

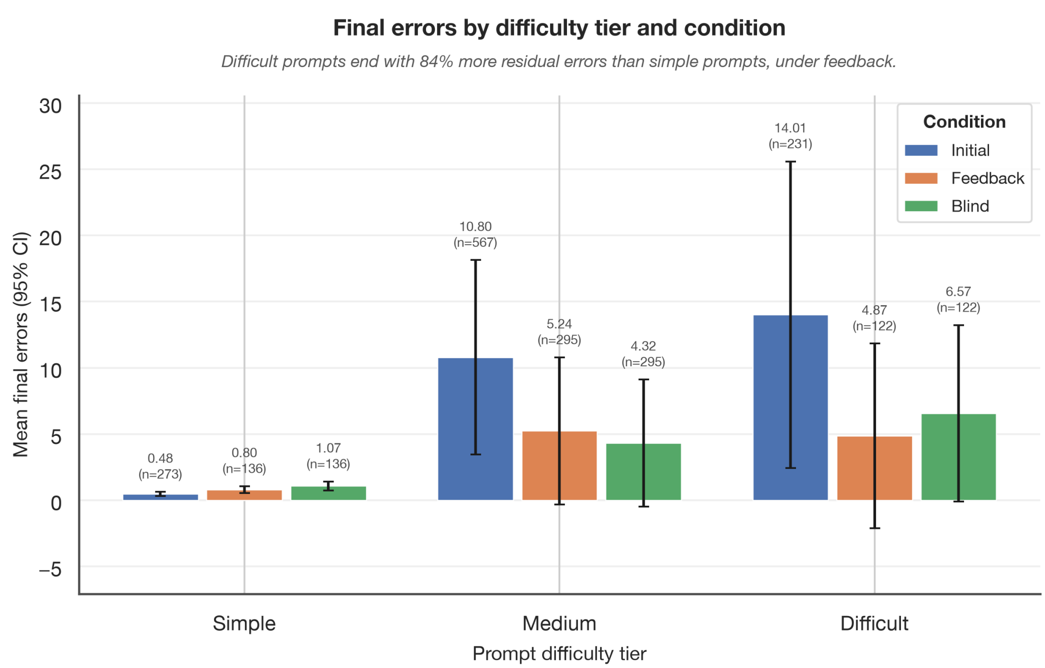

Figure 3: Mean final error count (95% CI) by prompt difficulty tier and condition. Value and n are annotated above each bar.

Discuss which error types the guardrail is most and least effective at resolving. For example: does it reliably fix missing-doctype errors? Does it struggle with disallowed-attribute errors? This dimension answers the qualitative question of what the guardrail actually corrects, and is important for contextualising the aggregate numbers. A small frequency table of error categories (before vs. after) serves this paragraph well. I think I can retrieve this data from the json files in validation/reprompt/, not sure though?

6.2 RQ2 — Cost-Efficiency

Briefly restate the scope: this is local-only comparison. Cost is operationalised as generation latency (`gen_time_s`) and token consumption (`prompt_eval_count` + `eval_count`) per run, not API pricing. Set the reader's expectation before presenting numbers

Present the main RQ2 finding: a scatter plot (or table) of each model's error reduction rate against its total generation time and token count. Colour-code or otherwise distinguish reasoning vs non-reasoning models. This is where the core finding lives – reasoning capability is more predictive of success than parameter count, with `codestral:22b` as the key counterexample (the most parameters, not the absolute best performance due to lack of reasoning -> a smaller model performed ever so slightly better).

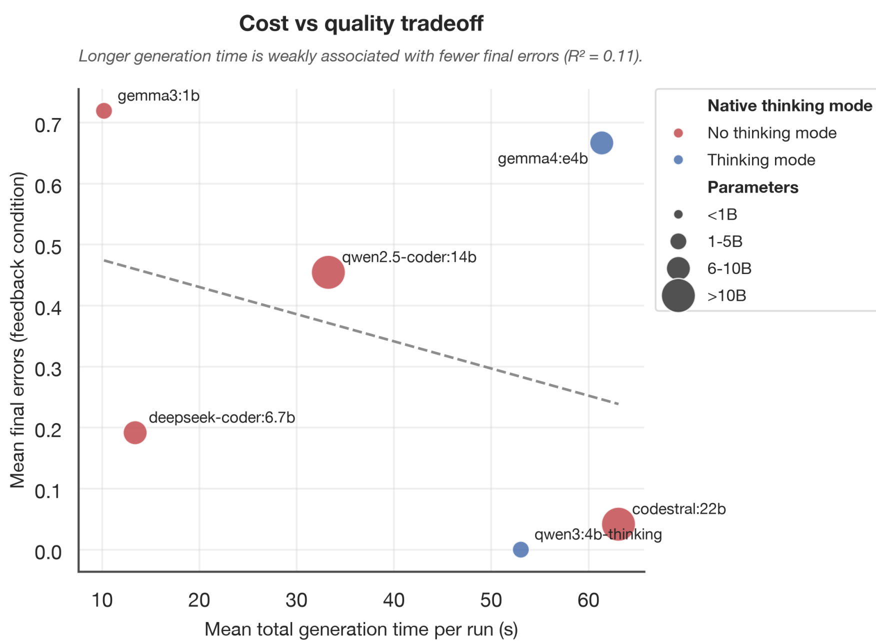

Figure 4: Mean total generation time per run vs mean final error count (feedback condition), one point per model. Colour encodes native thinking mode, point size encodes parameter-count bin.

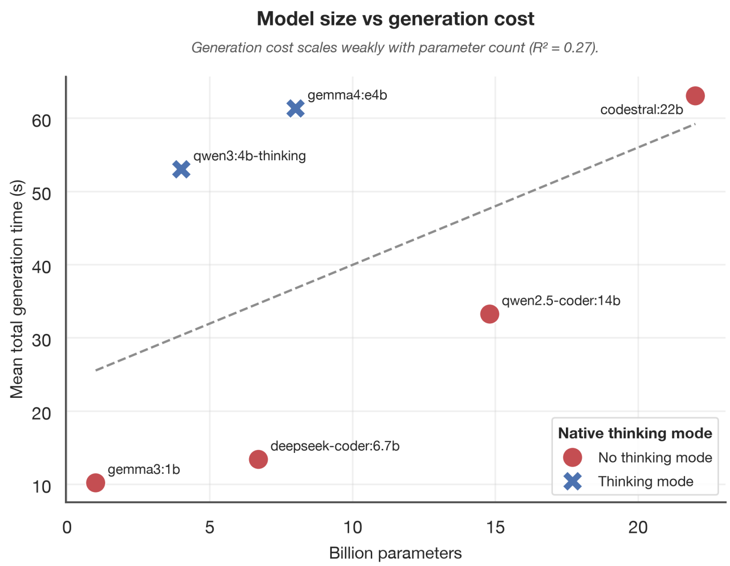

Figure 5: Parameter count vs mean total generation time per run, coloured by native thinking mode.

Give this finding its own paragraph because it is the most interesting and surprising result of RQ2. Explicitly compare a smaller reasoning model against codestral:22b. If a reasoning model with fewer parameters achieves better error reduction in fewer iterations at lower latency, that is a concrete, reportable conclusion that directly challenges the assumption that more parameters equal better output.

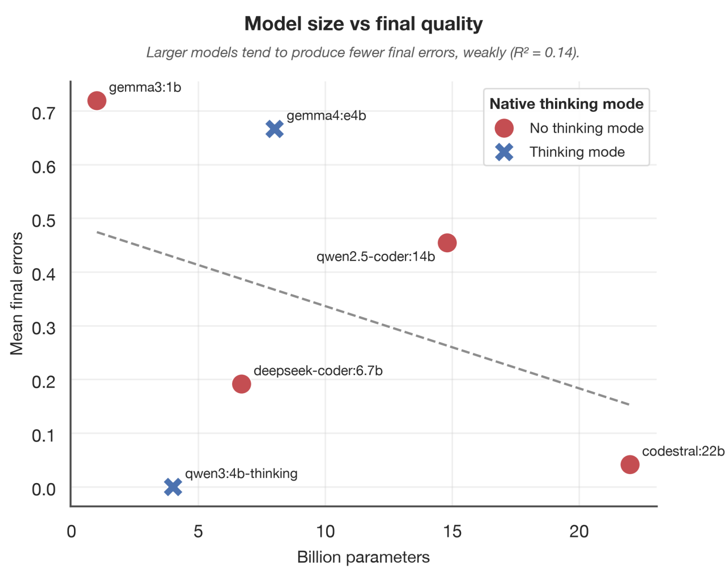

Figure 6: Parameter count vs mean final error count (feedback condition), coloured by native thinking mode. qwen3:4b-thinking (4B, reasoning) reaches a lower mean final error count than codestral:22b (22B, non-reasoning) at roughly a fifth of the parameters.

Does the `feedback` condition cost more than `blind` in token terms? It should, since it passes longer prompts (original code + error list). Is the extra cost justified by the quality gain? Frame this as a cost-benefit ratio: additional tokens spent per additional error resolved.

6.3 Iteration Count Analysis

Using the output of `stats.py`'s iteration-cutoff analysis, describe when models typically converge. Do most runs resolve within 1-2 iterations, or does error reduction continue meaningfully up to the maximum? Present a distribution (histogram or line chart) of iterations-to-convergence across all (model, prompt) pairs in the `feedback` condition.

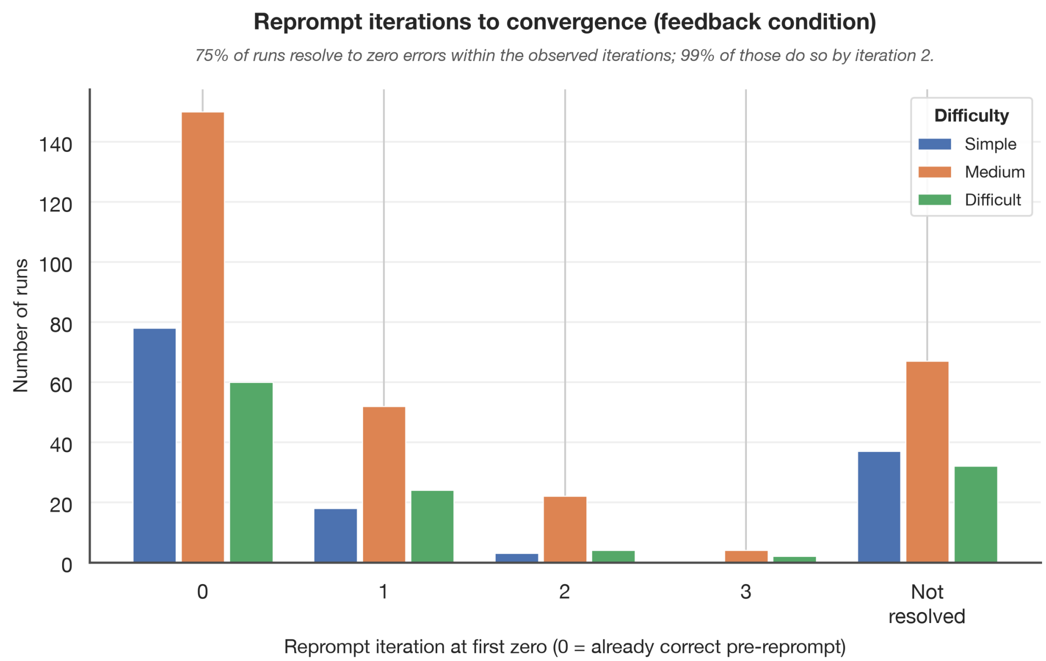

Figure 7: Distribution of the reprompt iteration at which each (model, prompt, trial) run first reaches zero errors, feedback condition, split by prompt difficulty tier. "Not resolved" = still nonzero at the last observed iteration.

Identify where the marginal gain per additional iteration drops below a meaningful threshold. If, for example, 80% of resolvable errors are fixed by iteration 2 and iterations 3-5 contribute minimally, that is your practical recommendation for the maximum iteration count. State this as a concrete finding rather than a vague observation. Note whether this threshold differs by model or difficulty tier.

7 Discussion (2 – 3 pages)

8 Conclusion (0.5 – 1 page)

References

[1] Alexandra Souly, Javier Rando, Ed Chapman, Xander Davies, Burak Hasircioglu, Ezzeldin Shereen, Carlos Mougan, Vasilios Mavroudis, Erik Jones, Chris Hicks, Nicholas Carlini, Yarin Gal, and Robert Kirk. Poisoning attacks on llms require a near-constant number of poison samples, 2025. URL `https://arxiv.org/abs/2510.07192`.

[2] Florian Tambon, Arghavan Moradi-Dakhel, Amin Nikanjam, Foutse Khomh, Michel Desmarais, and Giuliano Antoniol. Bugs in large language models generated code: an empirical study. *Empirical Software Engineering*, 30, 02 2025. doi: 10.1007/s10664-025-10614-4.

[3] Shihan Dou, Haoxiang Jia, Shenxi Wu, Huiyuan Zheng, Muling Wu, Yunbo Tao, Ming Zhang, Mingxu Chai, Jessica Fan, Zhiheng Xi, Rui Zheng, Yueming Wu, Ming Wen, Tao Gui, Xipeng Qiu, and Xuanjing Huang. What is wrong with your code generated by large language models? an extensive study. *Science China Information Sciences*, 69, 01 2026. doi: 10.1007/s11432-025-4632-8.

[4] Ravin Ravi, Dylan Bradshaw, Stefano Ruberto, Gunel Jahangirova, and Valerio Terragni. Llmloop: Improving llm-generated code and tests through automated iterative feedback loops. 07 2025. doi: 10.1109/ICSME64153.2025.00109.

[5] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Sean Welleck, Bodhisattwa Majumder, Shashank Gupta, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback, 03 2023.

[6] Tristan Coignion, Clément Quinton, and Romain Rouvoy. A performance study of llm-generated code on leetcode, 07 2024.

Appendix

Full prompt dataset

Extended results tables

Flowchart (if not in main body)

Human evaluation rubric (if used)

Project timeline

- Full prompt dataset
- Extended results tables
- Flowchart (if not in main body)
- Human evaluation rubric (if used)
- Project timeline — your supervisor asked for this; an appendix is a natural place for if it doesn't fit in the main body